Inferencia local de LLM:

Estrategias de optimización para la informática personal.

Autor: Oscar Gallardo
Afiliación: Proyecto lap-llm / Hackatón CubePath 2026
Fecha: Marzo 2026

Resumen

El acceso a modelos de lenguaje de gran escala (LLM) ha quedado tradicionalmente restringido a infraestructuras cloud o estaciones de trabajo de alto rendimiento. Este trabajo presenta un marco práctico para democratizar el acceso a la IA local mediante la ejecución de LLMs en hardware modesto — específicamente laptops con GPUs integradas (iGPUs) bajo sistemas Linux.

Proponemos una **arquitectura de tres capas** que integra:

1. **Optimización del kernel Linux y gestión de memoria unificada (UMA).**
2. **Backends de inferencia heterogéneos** (SYCL para Intel, ROCm/HIP para AMD o Vulkan).
3. **Estrategias de cuantización adaptativas.**

Demostramos que mediante la comprensión profunda de los mecanismos de memoria del kernel — incluyendo PPGTT, TTM, y políticas NUMA — es posible ejecutar modelos de **mas capaces en laptops con 8-32GB RAM** sin degradación crítica del rendimiento.

Nuestro enfoque prioriza la **soberanía digital**: el usuario mantiene control total sobre sus datos y dependencias tecnológicas, eliminando la necesidad de conectividad cloud o suscripciones a APIs propietarias. Este trabajo establece las bases técnicas para sistemas de asesoramiento automatizado de hardware, validando que la IA útil es accesible más allá de las fronteras del hardware premium.

---

1. Introducción

- **Contexto:** La centralización de la IA en grandes proveedores cloud crea dependencias tecnológicas y vulnerabilidades de privacidad. Paralelamente, la narrativa técnica asume erróneamente que la inferencia local de LLMs requiere GPUs dedicadas de alto consumo.
- **Problema:** Falta de documentación sistemática sobre cómo aprovechar arquitecturas de memoria unificada y backends de cómputo heterogéneo en hardware común.
- **Contribución:** Un marco técnico verificado para ejecutar LLMs capaces en laptops con iGPUs, con énfasis en optimizaciones a nivel de kernel y

gestión de memoria.

I.I. Fundamentos de Gestión de Memoria en Linux

La gestión de la memoria en sistemas GNU/Linux es un factor determinante para la viabilidad de la IA local. En equipos de características moderadas, como laptops o estaciones de trabajo sin una GPU dedicada, la configuración del kernel es la frontera entre la ejecución exitosa de un modelo o el fallo por falta de recursos (Out of Memory).

1. La UMA como Recurso Elástico

Si bien las unidades de cómputo (ALUs/Tensor Cores) son vitales para el procesamiento paralelo de tensores, la VRAM (Video RAM) es el recurso más crítico por tres razones fundamentales:

1. **Residencia del Modelo (Weights):** Los parámetros de un LLM deben cargarse íntegramente en la memoria de video para evitar el "cuello de botella" del bus PCIe. Si el modelo no cabe en VRAM, el sistema recurre al swapping o al uso de la CPU, reduciendo la velocidad de generación de tokens de forma drástica.
2. **KV Cache (Key-Value Cache):** Durante la inferencia, el sistema almacena las representaciones de los tokens previos para no recalcularlos en cada paso. A mayor contexto (8k, 32k, 128k tokens), mayor es la demanda de VRAM. Sin memoria suficiente, el "hilo de la conversación" se ve severamente limitado.
3. **Activaciones y Overheads:** Más allá del peso del modelo, el motor de inferencia requiere un margen operativo para las activaciones intermedias y la gestión de tensores temporales durante el proceso de forward pass.

2. Ventajas Competitivas de la Arquitectura UMA

La Arquitectura de Memoria Unificada (UMA) no es una innovación reciente, sino una evolución de sistemas de alto rendimiento que tiene sus raíces en las estaciones de trabajo de SGI y Sun Microsystems en los años 80. Este concepto, que permite a múltiples procesadores acceder a un pool de memoria compartido, se perfeccionó para el consumo masivo en 1996 con la arquitectura basada en MIPS de la Nintendo 64, eliminando la necesidad de copiar datos entre componentes. Con la llegada de los años 2000, esta capacidad se integró profundamente en el kernel Linux mediante el Direct Rendering Manager (DRM), permitiendo una gestión granular del swappiness y las páginas de memoria. A diferencia de las GPUs discretas o las implementaciones rígidas en otros sistemas operativos, la ventaja crítica de UMA en Linux es que permite trascender los límites estáticos asignados por la BIOS. En lugar de conformarse con un número rígido de VRAM, el sistema puede solicitar dinámicamente recursos

de la RAM física, transformando una supuesta limitación de las iGPUs en un recurso elástico y masivo, ideal para las exigentes demandas de memoria de los LLM contemporáneos.

A diferencia de las GPUs discretas, las iGPUs (Integrated GPUs) no poseen memoria física dedicada, lo cual suele verse como una limitación. Sin embargo, en el ecosistema Linux, esto se convierte en una ventaja mediante la Arquitectura de Memoria Unificada (UMA) gestionada a través del DRM (Direct Rendering Manager).

Dentro del stack del kernel, coexisten dos manejadores de memoria cruciales:

- **GEM (Graphics Execution Manager):** Optimizado para dispositivos UMA, permite una gestión simplificada de los objetos gráficos.
- **TTM (Translation Table Manager):** Un gestor más robusto que soporta tanto UMA como VRAM dedicada, facilitando el movimiento de datos entre diferentes tipos de memoria.

En una arquitectura UMA bien configurada, la iGPU y la CPU comparten la misma RAM física. Esto permite que stacks de software como AMD ROCm/HIP o Intel OneAPI accedan a un pool de memoria mucho más flexible que el de una tarjeta de video comercial de gama media.

El desafío crítico para Laptops: Por defecto, la mayoría de las BIOS/Firmwares reservan entre 500 MB y 2 GB de RAM para la iGPU. Para un LLM moderno, esto es insuficiente. La importancia de este proyecto radica en la capacidad de reconfigurar el kernel para expandir este límite, permitiendo asignar, por ejemplo del 50% al 90% de la RAM del sistema como memoria de video efectiva.

2. Componentes para la Gestion de Memoria en GNU/Linux.

En el ecosistema de Linux y la gestión de memoria para gráficos (DRM), GTT, PPGTT y TTM son componentes que permiten que la GPU acceda y organice la memoria (ya sea RAM compartida o VRAM dedicada) para ejecutar tareas complejas como la inferencia de modelos de lenguaje

TTM (Translation Table Manager) Es el primer gestor de memoria genérico desarrollado para el núcleo DRM de Linux, diseñado para ser una solución integral para diversos tipos de hardware y su trabajo es gestionar el ciclo de vida de los Buffer Objects (BO), encargándose de su creación, destrucción, mapeo a la CPU y, lo más importante, su movimiento entre diferentes tipos de memoria. TTM es capaz de manejar tanto dispositivos con memoria compartida (UMA) como tarjetas con memoria de video (VRAM) dedicada.

GTT (Graphics Translation Table) Es una tabla de traducción global mantenida por el kernel que mapea las direcciones virtuales de la GPU a direcciones físicas en la memoria del sistema y actúa como un puente único para

todo el hardware de video; en modelos antiguos (previos a la generación 8 de Intel), todos los procesos compartían esta misma tabla global.

PPGTT (Per-Process Graphics Translation Table) Es una evolución de la GTT donde cada contexto de la GPU (similar a un proceso de CPU) tiene su propia tabla de traducción independiente y proporciona un espacio de direcciones virtual único para cada cliente, garantizando el aislamiento y la seguridad entre aplicaciones. Esto evita que un proceso pueda acceder o corromper la memoria de otro.

I.II. Arquitectura de un LLM y su relacion con la VRAM de una GPU

Los LLM se benefician especialmente del uso de GPUs por dos razones principales. En primer lugar, por su capacidad de procesamiento paralelo: como se menciona en "Brook for GPUs: Stream Computing on Hardware Graphics" (p. 3), "un stream es una colección de datos que puede ser computada en paralelo". Esta característica resulta particularmente útil para las operaciones con matrices y tensores que emplean las redes neuronales y, en general, los LLM. En segundo lugar, las GPUs resultan ventajosas por la mayor velocidad de su VRAM —típicamente GDDR— en comparación con la RAM utilizada en equipos de cómputo convencionales como laptops o APUs, que suele ser de tipo DDR.

Las arquitecturas de los LLM han evolucionado con el objetivo de permitir contextos más grandes y, al mismo tiempo, lograr una mayor eficiencia computacional. Prácticamente cualquier modelo moderno se basa en el Transformer, una red neuronal diseñada para procesar secuencias de datos en paralelo. Un LLM se compone de una pila de bloques de transformador repetidos —por ejemplo, GPT-3 contiene 96 de estos bloques—.

Algunas sub capas importantes son: - **Mecanismo de autoatención:** permite que cada token en una secuencia se relacione con los demás para capturar el contexto mediante tres componentes: Query (Q, lo que se busca), Key (K, lo que cada token ofrece) y Value (V, la información real que se extrae). - **Feed-Forward Network (FFN):** procesa cada posición de forma independiente para capturar relaciones más complejas. - **Normalización de Capas (LayerNorm):** Estabiliza el entrenamiento asegurando que las salidas tengan una media y varianza consistentes.

Una característica particular de los mecanismos de autoatención es que no poseen conocimiento inherente del orden de las palabras en una secuencia. Para solucionar esto, se añaden embeddings posicionales a los embeddings de los tokens. Posteriormente, durante el muestreo de datos, se crean pares de entrada (X) y objetivo (Y): si la entrada es una secuencia de tokens, el objetivo es esa misma secuencia desplazada un token hacia la derecha. De este modo, el modelo intenta predecir el siguiente token en cada posición.

1. Multi-Head Attention (MHA)

Es la forma estándar de atención introducida en el artículo original "Attention is All You Need". En lugar de realizar una única operación de atención, el modelo divide las dimensiones en múltiples "cabezas" que operan en paralelo. Esto permite que el modelo atienda simultáneamente a diferentes tipos de relaciones en los datos. Sin embargo, es computacionalmente costoso y consume considerable memoria, especialmente durante la inferencia, debido al manejo del KV Cache.

2. Grouped-Query Attention (GQA)

GQA surge como una optimización crítica para modelos grandes, situándose entre MHA y Multi-Query Attention (MQA). Mientras que en MHA cada cabeza de Query posee su propia cabeza de Key (K) y Value (V), en GQA múltiples cabezas de Query comparten una única cabeza de K y V. Esto reduce drásticamente el tamaño del KV Cache, permitiendo manejar ventanas de contexto significativamente mayores. GQA ofrece una calidad cercana a la de MHA, pero con una velocidad de inferencia comparable a la de MQA (que utiliza una sola cabeza K/V para todo el modelo).

El KV Cache (Cache de Claves y Valores)

Durante la generación de texto (inferencia), el modelo predice un token a la vez de forma autorregresiva. Para predecir el siguiente token, necesita las representaciones (Keys y Values) de todos los tokens anteriores; recalcular estas matrices en cada paso sería extremadamente ineficiente. El KV Cache almacena estas matrices en memoria para su reutilización inmediata.

Mixture of Experts (MoE)

MoE es una técnica de escritura dispersa (sparse) que permite escalar modelos a billones de parámetros sin incrementar proporcionalmente el costo computacional. Reemplaza la capa densa de Feed-Forward por un conjunto de submodelos especializados llamados expertos. Una red de compuertas (gating network) determina, para cada token individual, qué experto(s) deben procesarlo —típicamente solo 1 o 2 expertos de entre 16, 64 o 256 disponibles—.

Para este experimento, las cuantizaciones resultaron fundamentales. Se trata de una técnica que permite reducir los requerimientos computacionales y de almacenamiento de las redes neuronales profundas. El proceso consiste en convertir los números de punto flotante —que suelen ser de alta precisión y representan los pesos y sesgos del modelo— en representaciones de menor precisión, como números enteros. Mediante la comprensión de estos conceptos, es posible seleccionar un modelo que se ajuste adecuadamente a las limitaciones del hardware disponible.

I.III. Marco Matemático para la Gestión de Memoria

Para garantizar la estabilidad de un sistema y evitar el OOM (Out of Memory) Killer, la asignación de memoria sera dividdida en tres etapas de cálculo:

A. Cálculo de Disponibilidad Dinámica (Steady State) Determina cuánta memoria física puede ser cedida a la GPU sin colapsar el sistema operativo.

$$VRAM_{usable} = RAM_{libre} - Margen_{seguridad}$$

Donde el Margen de seguridad se puede establecer en **1.5 GB** para operaciones críticas del Kernel, o en un 0.05% del total de la RAM.

B. Requerimiento Total de Inferencia ($VRAM_{req}$) Suma el peso estático del modelo y la memoria volátil del contexto.

$$VRAM_{req} = W_{model} + KV_{cache} + \delta$$

- W_{model}: 16.4 GB
- δ: Overhead de buffers (aprox. 1.0 GB)
- KV_{cache}: Se calcula según la arquitectura del modelo:

$$KV_{cache} \approx \frac{2 \times L \times H \times D \times C \times P}{1024^3}$$

(Para Qwen3-30B con 8k tokens y precisión FP16, este valor es de **1.5 GB**)

C. Conversión a Páginas del Kernel (TTM Pages) El parámetro `amdttm.pages_limit` requiere el valor en unidades de páginas (4 KB en x86_64).

$$TTM_{pages} = \frac{VRAM_{req} \times 1024 \times 1024}{4}$$

Ejemplo de validación para 30B (8k contexto):

$$VRAM_{req} = 16.4 + 1.5 + 1.0 = \mathbf{18.9} \text{ GB}$$

$$TTM_{pages} = \frac{18.9 \times 1,048,576}{4} = \mathbf{4,954,522} \text{ páginas}$$

D. Conversión a Páginas GTT (Graphics Translation Table) (Deprecated) GTT opera con marcos de página de 4 KB, la configuración del kernel debe expresarse siempre en unidades de página. La ecuación general de conversión, ajustada a la precisión de direccionamiento del kernel, es:

$$GTT_{pages} = \lfloor \frac{VRAM_{limit} \times 1024^2}{4} \rfloor$$

(Donde $VRAM_{limit}$ se expresa en MB para coincidir con los parámetros de modprobe).

II. Implementación y Optimización en iGPUs AMD

1. Entorno de Validación y Hardware

Para validar las estrategias de optimización, se utilizó una plataforma de hardware de consumo con las siguientes especificaciones: - **Modelo** Thinkpad T14s - **CPU**: AMD Ryzen 7 PRO 4750U (8C/16T, Zen 2) - **iGPU**: AMD Radeon Vega 8 (Arquitectura nativa gfx90c) - **RAM**: 32GB DDR4 (~29GB usables para el sistema) - **OS**: Gentoo Linux x86_64 con kernel 6.18.12 personalizado

Nota sobre disponibilidad de memoria: En un sistema minimalista (consumo base entre 100MB~600MB), la memoria asignable es de considerablemtente mayor, mientras que en distribuciones comerciales (Ubuntu, Fedora), el sistema suele consumir entre 2GB y 4GB, lo que reduce el margen disponible para el modelo. En mi sistema gentoo sistemas me quedaban 20GB y 24GB libres para asignar.

2. Configuración del Kernel: Ampliando el Límite de VRAM

Para ejecutar un modelo como Qwen3-Coder-30B (cuantización IQ4_XS), se requiere que la iGPU pueda direccionar cerca de 20GB. Esto se logra modificando el parámetro del driver de memoria de AMD (amdttm). **Arquitectura del Modelo:** Qwen3-30B-A3B

El despliegue de un modelo de 30 mil millones de parámetros en una laptop es posible gracias a la arquitectura Mixture-of-Experts (MoE) y técnicas de cuantización avanzada.

Especificación	Valor	Implicación Técnica
Parámetros Totales	30B	Alta capacidad de razonamiento y conocimiento
Parámetros Activos	~3B	Carga computacional reducida por token (baja latencia)
Cuantización	IQ4_XS	Reducción de precisión a ~4 bits con mínima pérdida

Especificación	Valor	Implicación Técnica
Peso en Disco (W_{model})	16.4 GB	Tamaño real del archivo GGUF mapeado en memoria

Cálculo de Páginas El parámetro `amdttm.pages_limit` recibe valores en páginas de memoria (4KB). Para este experimento hubo que asignar 20GB:

```
20 GB × 1024 × 1024 / 4 = 5,242,880 páginas
```

Métodos de Aplicación Dependiendo de tu nivel de experiencia, elige uno de estos tres métodos:

A. Mediante GRUB (Recomendado): Edita `/etc/default/grub` y añade los parámetros a `GRUB_CMDLINE_LINUX_DEFAULT`:

```
GRUB_CMDLINE_LINUX_DEFAULT="... amdttm.pages_limit=5242880 amdttm.page_pool_size=524288"
```

Luego, actualiza la configuración:

- **Debian/Ubuntu/Mint**: `sudo update-grub`
- **Fedora/Arch/Gentoo**: `sudo grub-mkconfig -o /boot/grub/grub.cfg`

B. Mediante EFIBOOTMGR (Arranque Directo):

```
sudo efibootmgr --create --disk /dev/nvme0n1 --part 1 --label "Lap-LLM Linux" --loader /vmli
```

C. Compilación Manual (Gentoo): Añadir directamente al `CONFIG_CMDLINE` en la configuración del kernel.

3. Preparación del Entorno ROCm e ISA (Arquitectura)

Las iGPUs de laptop suelen tener arquitecturas no soportadas oficialmente por AMD (como gfx90c). La solución es forzar el uso de una arquitectura compatible de la misma familia (ISA).

Identificación de Arquitecturas comunes:

- **Familia GFX9 (Vega)**: gfx90c (Laptops Ryzen 4000/5000). Se debe usar el override o compilar para gfx900.
- **Familia GFX10.3 (RDNA 2)**: gfx1031/1035 (Ryzen 6000/7000). Se usa gfx1030.
- **Familia GFX11 (RDNA 3)**: gfx1103 (Ryzen 7000/8000).

Para conocer tu arquitectura exacta, instala y ejecuta:

```
rocminfo | grep gfx
```

Instalar ROCm Core:

Dependiendo de la distros, y el gestor de paquetes el comando sera distinto. se recomienda checar el sitio oficial para buscar tu distro. Quick start installation guide

Instalación de un Local LLM Runtime:

Existen diversas opciones para ejecutar modelos en Linux. La elección depende de si buscas facilidad de uso o control total sobre el binario.

Opción A: Instalación Automatizada (Recomendado) Es la vía más rápida. Los instaladores detectan la mayoría de las configuraciones de hardware automáticamente, aunque generan binarios más pesados al incluir soporte para múltiples arquitecturas.

- **Ollama**

```
curl -fsSL https://ollama.com/install.sh | sh
```

- **LM Studio**

Permite una interfaz gráfica (GUI) avanzada para gestionar modelos.

```
curl -fsSL https://lmstudio.ai/install.sh | bash
# O mediante SDK:
npm install @lmstudio/sdk
```

Opción B: Compilación desde Fuente (Optimización para Expertos) Compilar manualmente permite generar un binario específico para tu ISA (ej. gfx900), eliminando el código muerto de otras arquitecturas y mejorando la eficiencia en sistemas con recursos limitados.

- **Ollama con codigo fuente.**

Obtención del código:

```
git clone https://github.com/ollama/ollama.git
cd ollama
```

Configuración de Arquitectura (ISA Tuning):

Edita el archivo `CMakePresets.json`. En la sección "ROCm 6", limpia la variable `GPU_TARGETS` para dejar únicamente la arquitectura compatible con tu iGPU. Esto evita compilar componentes innecesarios.

```
"cacheVariables": {
  "CMAKE_HIP_FLAGS": "",
  "GPU_TARGETS": "gfx900", // Sustituye por tu ISA objetivo
  "OLLAMA_RUNNER_DIR": "rocm"
}
```

Construcción del binario:

```
# Compilar los componentes de C++ (Runner)
cmake --preset "ROCm 6"
cmake --build --preset "ROCm 6"

# Compilar el binario principal en Go
go build -v -o build/bin/ollama .
```

Ejecución y Validación del Offloading

Una vez instalado el motor, es crucial asegurarse de que el modelo se cargue en la "VRAM" elástica que configuramos en el kernel.

Si utilizas una iGPU no soportada oficialmente (como la gfx90c), debes ejecutar el servicio con el override de versión para que el runtime de ROCm acepte el hardware:

```
export HSA_OVERRIDE_GFX_VERSION=9.0.0
./build/bin/ollama serve
```

Nota de Estabilidad: Es normal experimentar un "freezzeo" momentáneo la primera vez que se cargan los pesos del modelo a la memoria compartida (offloading). Si el sistema reinicia o se bloquea permanentemente, revisa que el Margen_seguridad de la RAM no sea demasiado bajo para el kernel.

II.I Resultados: Comparativa de Capacidades

La transición de una gestión basada en BIOS a una gestión basada en el subsistema TTM del Kernel permite un salto generacional en la capacidad de inferencia local.

Parámetro	Configuración Estándar (BIOS)	Configuración Optimizada (TTM)
Método de Reserva	UMA Framebuffer (Estático)	amdttm.pages_limit (Dinámico)
Límite de VRAM	2 GB fijos	20 GB+ (según disponibilidad)
Modelo Máximo	~1.5B (Qwen2.5 1.5B)	30B (Qwen3-30B-A3B)
Contexto Viable	2,048 tokens	8,192+ tokens
Impacto en RAM	2GB bloqueados permanentemente	0GB en reposo; se asigna al cargar el modelo

III. Estrategias para iGPUs Intel: Estado del Arte e Hipótesis de Investigación

Nota metodológica: Esta sección se basa en análisis de documentación técnica, reportes de la comunidad y arquitectura de drivers, dado que el trabajo experimental se realizó en plataforma AMD. Se propone una hipótesis operativa para investigación futura.

1. Backend de Inferencia: SYCL sobre OpenCL

Para iGPUs Intel, la opción técnicamente madura es llama.cpp con backend SYCL, abandonando OpenCL por limitaciones de rendimiento.

Aspecto	Especificación
Requisito mínimo	Intel Core 11ª generación (arquitectura Xe)
Stack de software	Intel oneAPI Base Toolkit (compilador SYCL + oneMKL)
Ventaja clave	Vectorización eficiente de operaciones de atención
Limitación observada	Soporte discontinuado para arquitecturas Gen9 y anteriores

Comparativa con AMD: Mientras AMD requiere compilación de ROCm con targets específicos (gfx900), Intel proporciona oneAPI como distribución binaria, reduciendo la fricción inicial pero limitando la optimización a bajo nivel.

2. Gestión de Memoria: PPGTT vs. TTM

La divergencia arquitectónica entre Intel y AMD se manifiesta críticamente en el subsistema de memoria GPU:

Característica	AMD (TTM)	Intel (PPGTT + GEM)
Subsistema	Translation Table Manager	Per-Process Graphics Translation Table + Graphics Execution Manager
Control de pool	`amdttm.pages_limit` (parámetro kernel)	No existe equivalente directo documentado
Naturaleza	Pool paginado pre-reservable	Mapeo virtual por contexto
Límite práctico	Configurable hasta ~80% RAM	~50% RAM física (reportado)

Característica	AMD (TTM)	Intel (PPGTT + GEM)
Granularidad	Páginas de 4KB asignables bajo demanda	Buffer Objects (BO) gestionados por GEM

El problema del 50% en Intel

Documentación de la comunidad y manuales de referencia indican que iGPUs Intel en Linux típicamente operan con un techo del 50% de RAM física asignable como memoria gráfica. En un sistema con 32GB RAM, esto implica ~16GB máximo — suficiente para modelos 7B-13B, pero restrictivo comparado con los 20GB logrados vía TTM en AMD.

Estrategia documentada para mitigar: Aumento físico de RAM a 32GB, permitiendo que el 50% asignable (~16GB) sea funcionalmente equivalente al límite TTM configurado en sistemas AMD con menos RAM total.

3. Hipótesis: Hacia un "PPGTT Ballooning"

Basado en el análisis de arquitectura de drivers y técnicas de virtualización (Intel GVT-g), se plantea la siguiente hipótesis de investigación:

Hipótesis: Es posible extender el límite efectivo de memoria GPU en iGPUs Intel mediante la manipulación de parámetros del driver i915 relacionados con el espacio de direcciones PPGTT, o mediante técnicas de "address space ballooning" adaptadas de entornos virtualizados.

Fundamento técnico: PPGTT proporciona aislamiento de direcciones virtuales por contexto, no reserva física. Sin embargo, el driver GEM subyacente asigna páginas físicas de memoria del sistema (shmem) a Buffer Objects bajo demanda. La barrera del 50% no es hardware en la mayoría de implementaciones Linux, sino una heurística del driver para evitar contención con el sistema operativo.

Lineas de investigación propuestas:

Enfoque	Mecanismo	Estado
Boot parameter	`i915.enable_ppgtt=2` (Full PPGTT) + modificadores de límite	No documentado para expansión de memoria
Debugfs	Escritura en `/sys/kernel/debug/dri/*/i915_gem_objects`	Requiere privilegios, formato no estandarizado
Cgroups v2	Limitación inversa: restringir memoria del sistema para forzar asignación GPU	Hipótesis no validada

Enfoque	Mecanismo	Estado
Virtualización	Técnicas GVT-g de "ballooning" adaptadas a bare-metal	Complejidad alta, portabilidad dudosa

Diferencia fundamental con AMD:

```
AMD TTM:   Gestiona POOL de páginas físicas pre-asignables
           → amdttm.pages_limit controla el tamaño del pool

Intel GEM: Gestiona BO (Buffer Objects) mapeados vía PPGTT
           → El "pool" es la memoria del sistema menos usada por CPU
           → No existe parámetro directo para forzar reserva mínima/máxima
```

4. Configuración Recomendada (Basada en Documentación)

Para usuarios con iGPUs Intel que busquen replicar los resultados AMD sin modificaciones de kernel:

Componente	Recomendación	Rationale
RAM física	32GB mínimo	Eleva el 50% asignable a 16GB efectivos
Swap	32GB en SSD NVMe	Permite mmap de modelos grandes sin OOM
Cuantización	INT4 obligatorio (IQ4_XS/Q4_0)	Reduce 7B dense a ~4GB, 7B MoE a ~3GB activos
Contexto	4096-6144 tokens	Balance entre capacidad y presión de KV cache
Herramienta	Ollama + llama.cpp (SYCL) o BigDL-LLM	Abstracción de complejidad oneAPI

5. Trabajo Futuro: Validación de la Hipótesis PPGTT

Se propone como línea de investigación inmediata:

1. **Investigación de parámetros i915:** Revisión exhaustiva de parámetros de módulo no documentados en `modinfo i915`
2. **Instrumentación del kernel:** Uso de ftrace y perf para mapear rutas de asignación GEM bajo carga de llama.cpp
3. **Prueba de concepto:** Modificación de `drivers/gpu/drm/i915/gem/i915_gem_shmem.c` para eliminar o parametrizar la heurística del 50%

4. **Comparativa SYCL vs. ROCm:** Benchmark de throughput (tokens/segundo) en hardware comparable (Core i7-1165G7 vs. Ryzen 7 PRO 4750U)

6. Conclusión de Sección

Mientras que en AMD el subsistema TTM permite configuración explícita del pool de memoria GPU mediante parámetros de kernel, en Intel la gestión es implícita a través de GEM y limitada por heurísticas del driver. La hipótesis de un "PPGTT ballooning" permanece sin validar, siendo la estrategia operativa actual la maximización de RAM física para elevar el límite del 50% a valores funcionalmente equivalentes.

Bibliografía

1. Drepper, U. (2007, 21 de septiembre). *What every programmer should know about memory*. Red Hat.
2. Gale, T. (2025, 3 de abril). *Efficient Mixture of Experts models*. Google DeepMind.
3. Grootendorst, M. (s.f.). *A Visual Guide to Mixture of Experts (MoE)*. Recuperado de https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-mixture-of-experts
4. *The Translation Table Manager (TTM)*. (s.f.). Recuperado de https://www.kernel.org/doc/html/latest/gpu/drm-mm.html
5. *Run LLMs on Intel® GPUs Using llama.cpp*. (s.f.). Intel. Recuperado de https://www.intel.com/content/www/us/en/developer/articles/technical/run-llms-on-gpus-using-llama-cpp.html
6. *drm/i915 Intel GFX Driver*. (s.f.). Recuperado de https://www.kernel.org/doc/html/latest/gpu/i915.html
7. Geerling, J. (2025). *Increasing VRAM allocation on AMD AI APUs under Linux*. Recuperado de https://www.jeffgeerling.com/blog/2025/increasing-vram-allocation-on-amd-ai-apus-under-linux/
8. Funnell, J. (s.f.). *Industrial LLM inference on a student's laptop?* Medium. Recuperado de https://medium.com/@john.funnell/industrial-llm-inference-on-a-students-laptop-a090a1bd1b2c